

Kacper

Osoba 3 - Kamera, oświetlenie, tekstury, cele

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Camera.*, Engine/Light.*, Engine/Texture.*, Demo/Targets.cpp, assets/textures/

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Kacper).

```
pgk/
|
|-- Engine/
|   |-- Math3D.* SceneNode.*          (Lukasz)
|   |-- PrimitiveNode.*              (Rogert)
|   |-- Camera.h / Camera.cpp <<< MOJE (kamera FPP i orbita)
|   |-- Light.h / Light.cpp <<< MOJE  (swiatlo + material Phong)
|   |-- Texture.h/ Texture.cpp <<< MOJE (BMP + tekstury proceduralne)
|   |-- Engine.*                     (Jakub)
|
|-- Demo/
|   |-- Room.cpp                     (Lukasz)
|   |-- Obstacles.cpp                (Rogert)
|   |-- Targets.cpp <<< MOJE (cele, fale, ruch celow)
|   |-- Gameplay.cpp Constants.h ... (Jakub)
|
+-- assets/textures/*.bmp <<< MOJE (pliki tekstur + instrukcja)
```

W skrocie: ja zajmuje sie tym, jak scena **wyglada** i jak **patrzemy** na nia. Kamera (skad i w ktora strone), oświetlenie (model Phong), tekstury (wczytane z BMP albo wygenerowane wzorami). W grze robie **cele** - ich pojawianie sie, ruch i animacje.

1. O co chodzi w moich plikach (po ludzku)

Kamera decyduje, co widzimy. Mam dwa tryby: 'orbita' (kamera krazy wokół punktu) i 'pierwsza osoba' (FPP - oko stoi w miejscu, a my rozglądamy się myszka). W grze używamy FPP.

Światło i **material** razem dają realistyczne cieniowanie (model Phong: ambient + diffuse + specular).

Tekstury to obrazki nalepiane na obiekty - albo wczytane z pliku BMP (sam napisałem parser), albo wygenerowane algorytmem (szachownica, drewno, cegły, marmur, szum).

W **Targets.cpp** robię cele do strzelania: losowe pojawianie się, ruch tam i z powrotem, obrót i kołysanie.

2. Camera - dwa tryby patrzenia

Kamera musi wyprodukować **macierz widoku** - mówi OpenGL, gdzie jest oko i w którą stronę patrzy. W trybie FPP liczę kierunek patrzenia z dwóch kątów: **yaw** (obrot lewo-prawo) i **pitch** (gora-dół).

Camera.cpp - kierunek patrzenia i macierz widoku

```
Vec3 Camera::lookDirection() const {
    if (cameraMode == Mode::FIRST_PERSON) {
        return Vec3( sin(yaw)*cos(pitch),    // X
                    sin(pitch),              // Y (gora/dol)
                    -cos(yaw)*cos(pitch) ); // Z (do przodu = -Z)
    }
    return normalize(orbitTarget - eyePosition()); // tryb orbity
}

Mat4 Camera::viewMatrix() const {
    Vec3 eye = eyePosition();
    Vec3 center = (cameraMode == Mode::FIRST_PERSON)
        ? (eye + lookDirection()) // FPP: patrz przed siebie
        : orbitTarget;           // orbita: patrz na cel
    return Mat4::lookAt(eye, center, Vec3(0,1,0));
}
```

Funkcję lookAt napisał Lukasz - ja jej używam, podając pozycję oka i punkt, na który patrzymy.

Ograniczam pitch do +/- 85 stopni, żeby nie dało się 'przewrócić' kamery do góry nogami.

3. Light - oświetlenie Phong

Korzystam z oświetlenia OpenGL (GL_LIGHT0..7). Material ma 3 składniki:

- **ambient** - kolor w cieniu (światło otoczenia, zawsze obecne),
- **diffuse** - główny kolor obiektu pod światłem rozproszonym,
- **specular** - błysk/odbicie, plus **shininess** mówi jak ostry jest błysk.

Light.cpp - ustawienie światła w danej pozycji sceny

```
void PointLight::renderSelf(const Mat4& worldMatrix) const {
    GLenum lightId = GL_LIGHT0 + activeLightIndex;
    if (!isEnabled) { glDisable(lightId); return; }

    glPushMatrix();
    glMultMatrixf(worldMatrix.data());
    GLfloat position[4] = { 0,0,0, 1.0f }; // 1.0 = światło punktowe
    glEnable(lightId);
    glLightfv(lightId, GL_POSITION, position);
    glLightfv(lightId, GL_DIFFUSE, diffuse);
    // ... ambient, specular, attenuation ...
    glPopMatrix();
}
```

Pozycja światła to (0,0,0) **po** nałożeniu macierzy świata wezła - czyli światło świeci z miejsca, gdzie stoi jego wezeł. Czwarta współrzędna = 1 oznacza światło punktowe (gdyby 0, byłoby kierunkowe jak słońce).

Attenuation to tłumienie - im dalej, tym ciemniej, według wzoru $1/(k_c + k_l \cdot d + k_q \cdot d^2)$.

Ważne: światła muszą być w grafie sceny **przed** geometrią, bo renderRecursive idzie po kolei. Gdyby ściana była przed światłem, nie zostałaby nim oświetlona.

4. Texture - wczytywanie i generowanie obrazkow

4.1. Wlasny parser BMP

Napisałem ręczny loader 24-bitowych BMP. Czyta nagłówki bajt po bajcie, sprawdza format i przerabia piksele. Dwie pułapki BMP:

- piksele są w kolejności **BGR** (nie RGB) - trzeba zamienić miejscami R i B,
- wiersze są zapisane **od dołu do góry** - trzeba je odwrócić,
- każdy wiersz jest **dosztukowany do wielokrotności 4 bajtów** (padding).

Texture.cpp - sedno parsera BMP

```
// konwersja BGR -> RGB z odwróceniem wierszy
for (int row = 0; row < h; ++row) {
    int srcRow = topDown ? row : (h - 1 - row); // odwrócenie
    for (int col = 0; col < w; ++col) {
        int src = srcRow * rowStride + col * 3; // rowStride = padding 4B
        int dst = (row * w + col) * 3;
        pixels[dst+0] = raw[src+2]; // R (z pozycji B)
        pixels[dst+1] = raw[src+1]; // G
        pixels[dst+2] = raw[src+0]; // B (z pozycji R)
    }
}
```

4.2. Tekstury proceduralne (bez plików)

Mam 7 generatorów: szachownica, gradient, paski, szum (Perlin/FBM), drewno, cegły, marmur. To czysta matematyka na pikselach. Przykład - drewno to koncentryczne kręgi zaburzone szumem:

Texture.cpp - generateWood (słoje drewna)

```
float turb = fbm2D(fx*2.5f, fy*2.5f, 4, seed) - 0.5f; // zaburzenie
float dist = sqrt(fx*fx + fy*fy) + turb*0.20f; // odległość od środka
float ring = dist * rings;
float fract = ring - floor(ring); // pozycja w obrębie słoja (0..1)
// jasne tło, ciemne słoje - interpolacja koloru wg fract
```

fbm2D (Fractional Brownian Motion) sumuje kilka 'oktaw' szumu o coraz drobniejszej skali - daje naturalny, samopodobny wzór. To moja własna implementacja value noise (losowe wartości na siatce + płynna interpolacja smoothstep).

4.3. Własne mipmapy

Mipmapy to pomniejszone wersje tekstury (1/2, 1/4, ...). OpenGL używa ich dla obiektów daleko, żeby nie migotały. Generuje je ręcznie - każdy poziom to усреднение bloków 2x2 z poprzedniego.

Texture.cpp - ręczne generowanie mipmap

```
while (mipW > 1 || mipH > 1) {
    int newW = max(1, mipW/2), newH = max(1, mipH/2);
    // dla każdego piksela: średnia z 4 pikseli (2x2) z wyższego poziomu
    glTexImage2D(GL_TEXTURE_2D, level, GL_RGB, newW, newH, 0,
                 GL_RGB, GL_UNSIGNED_BYTE, dst.data());
    mipW = newW; mipH = newH; ++level;
}
```

5. Demo/Targets.cpp - cele do strzelania

Cele to sfery, które pojawiają się w falach po 1-3 sztuki. Trzymam **pule** 3 gotowych sfer i tylko chowam/pokazuje je (setVisible), zamiast tworzyć i niszczyć - to szybsze i prostsze.

5.1. Ruch celu

Targets.cpp - pozycja celu liczona z sinusa (ruch ping-pong)

```
Vec3 ShootingGallery::currentTargetPos(const Target& t) const {
    Vec3 pos = t.basePos;
    if (t.moveRange > 0.01f)    // cel rusza się tam i z powrotem
        pos += t.moveAxis * (t.moveRange * sin(totalTime_*t.moveSpeed + t.movePhase));
    pos.y += 0.18f * sin(t.bobPhase * 2.2f);    // delikatne kołysanie w pionie
    return pos;
}
```

Sinus daje płynny ruch tam i z powrotem. **movePhase** jest losowe, żeby cele nie ruszały się zsynchronizowane. Ta sama funkcja jest używana przy rysowaniu i przy strzale - dzięki temu strzał trafia tam, gdzie cel naprawdę jest w danej chwili.

5.2. Bezpieczne pojawianie

Przy losowaniu pozycji celu sprawdzam: czy nie jest za blisko gracza, czy nie jest w środku przeszkody (używam isInsideObstacle Rogerta, z marginesem na cały zakres ruchu) i czy nie nachodzi na inny cel. Probuje do 60 razy, zanim dam za wygraną.

6. Pytania od wykładowcy - przygotowanie

Przy każdym pytaniu jest pasek **Gdzie** - mówi w którym pliku i w której funkcji szukać odpowiedzi, żebyś w trakcie obrony szybko otworzył właściwe miejsce.

Kamera - tryby

P: Czym różni się tryb orbity od pierwszej osoby?

O: W orbicie oko krąży wokół stałego punktu (orbitTarget) - dobre do oglądania obiektu. W FPP oko stoi w miejscu (fpEye), a yaw/pitch obracają kierunek patrzenia - jak w grach FPS. W grze używamy FPP. Tryb trzyma pole cameraMode.

Gdzie: Camera.h -> enum Mode; Camera.cpp -> eyePosition(), viewMatrix()

P: Jak z kątów yaw i pitch liczysz kierunek patrzenia?

O: To współrzędne sferyczne: $X = \sin(\text{yaw}) \cdot \cos(\text{pitch})$, $Y = \sin(\text{pitch})$, $Z = -\cos(\text{yaw}) \cdot \cos(\text{pitch})$. Przy yaw=0, pitch=0 patrzymy w -Z (do przodu w OpenGL). yaw obraca w poziomie, pitch w pionie. $\cos(\text{pitch})$ skaluje składowe poziome, żeby cały wektor miał długość 1.

Gdzie: Camera.cpp -> lookDirection() (głaz FIRST_PERSON)

P: Jak liczysz pozycję oka w trybie orbity?

O: $\text{eye} = \text{target} + (\cos(\text{pitch}) \cdot \sin(\text{yaw}), \sin(\text{pitch}), \cos(\text{pitch}) \cdot \cos(\text{yaw})) \cdot \text{distance}$. Czyli punkt na sferze o promieniu distance wokół targetu, wyznaczony przez dwa kąty. W FPP po prostu zwracam fpEye.

Gdzie: Camera.cpp -> eyePosition()

P: Jak budujesz macierz widoku i czym różni się center w obu trybach?

O: Wolam `Mat4::lookAt(eye, center, up)`. W FPP center = eye + lookDirection() (patrzymy przed siebie). W orbicie center = orbitTarget (patrzymy na stały punkt). Sama lookAt to kod Lukasa.

Gdzie: Camera.cpp -> viewMatrix(); Math3D.cpp -> Mat4::lookAt()

P: Dlaczego ograniczasz pitch do +/- 85 stopni?

O: Żeby nie dało się patrzeć idealnie w gore/dol i 'przekrecić' kamery do góry nogami. Przy 90 stopniach wektor patrzenia stałby się równoległy do osi 'up', co psuje lookAt - `cross(forward, up)` dałoby wektor zerowy (degeneracja).

Gdzie: Engine.cpp -> handleMouseMotion() (clamp pitch); ograniczenie dotyczy yaw/pitch silnika

P: Czemu w setOrbit/setFirstPerson zmieniasz cameraMode?

O: Bo te metody jednoznacznie określają tryb. Jak ktoś woła setFirstPerson, na pewno chce FPP - więc ustawiam cameraMode=FIRST_PERSON. To zapobiega pomyłce, gdzie zostałby stary tryb i kamera liczyłaby widok według złych pól.

Gdzie: Camera.cpp -> setOrbit(), setFirstPerson()

Oświetlenie - model Phong

P: Wytłumacz model Phong'a (3 składniki).

O: Kolor punktu to suma: ambient (stałe światło otoczenia, widoczne nawet w cieniu), diffuse (zależny od kąta między normalną a kierunkiem światła - rozproszenie, główny kolor) i specular (jasny błysk odbicia, ostrość sterowana przez shininess). OpenGL liczy to za nas, my podajemy parametry materiału (glMaterialfv) i światła (glLightfv).

Gdzie: Light.h -> struct Material; Light.cpp -> renderSelf(); PrimitiveNode.cpp -> glMaterialfv

P: Co to jest shininess i jak wpływa na błysk?

O: To wykładnik w części specular modelu Phong'a. Mały shininess = szeroki, rozmyty błysk (mat). Duży = mały, ostry punkt światła (polysk, jak metal/plastik). Ustawiam go per materiał - np. cele mają wysoki, ściany niski.

Gdzie: Light.cpp -> glMaterialf(..., GL_SHININESS, ...); wartości w Material

P: Co oznacza czwarta współrzędna pozycji światła (w=1)?

O: w=1 to światło punktowe - ma konkretną pozycję i świeci we wszystkie strony. w=0 to światło kierunkowe (jak słońce, w nieskończoności) - liczy się tylko kierunek, nie pozycja. My używamy punktowych, więc position[3] = 1.0f.

Gdzie: Light.cpp -> renderSelf() (GLfloat position[4] = {0,0,0, 1.0f})

P: Czemu pozycja światła to (0,0,0)? Przecież lampa jest gdzieś w pokoju.

O: Bo najpierw robie glMultMatrixf(worldMatrix), która przenosi układ do miejsca węzła światła. Wtedy (0,0,0) w tym układzie = faktyczna pozycja lampy w świecie. OpenGL przekształca pozycję światła aktualną macierzą modelview, więc to działa.

Gdzie: Light.cpp -> renderSelf() (glMultMatrixf przed glLightfv POSITION)

P: Co to jest attenuation (tłumienie) i jaki ma wzór?

O: Spadek jasności z odległości: $1/(k_c + k_l \cdot d + k_q \cdot d^2)$, gdzie d to odległość od światła, k_c stała, k_l liniowa, k_q kwadratowa. Mniejsze współczynniki = światło sięga dalej. Ustawiamy małe wartości, żeby cały pokój był równomiernie oświetlony.

Gdzie: Light.cpp -> renderSelf() (GL_CONSTANT/LINEAR/QUADRATIC_ATTENUATION)

P: Dlaczego światła muszą być dodane do sceny przed geometrią?

O: Bo renderRecursive (Lukasz) przechodzi węzły po kolei i ustawia stan OpenGL na bieżąco. Światło ustawia glLight dopiero gdy do niego dojdziemy. Geometria narysowana wcześniej nie 'widzi' jeszcze tego światła. Dlatego w grze dodaje światła jako pierwsze dzieci roota.

Gdzie: Light.cpp -> renderSelf(); kolejność w Gameplay.cpp (addChild światel najpierw)

P: Jak obsługuje wiele swiateł naraz?

O: Kazde swiatlo ma `activeLightIndex` (0..7), z ktorego licze `GLenum lightId = GL_LIGHT0 + index`. Dzieki temu dwie lampy (index 0 i 1) nie nadpisuja sie - kazda steruje innym `GL_LIGHTx`. OpenGL obsluguje do 8 swiateł jednocześnie.

Gdzie: `Light.cpp` -> `renderSelf() (GL_LIGHT0 + activeLightIndex); setLightIndex()`

Tekstury - parser BMP

P: Jakie formaty BMP obsluguje twoj loader?

O: Tylko 24-bitowe nieskompresowane (`BI_RGB`, `kompresja=0`). Sprawdzam to czytając z naglowka DIB `bpp` (bits per pixel) i `compression`. Jezeli to nie 24-bit bez kompresji, zwracam `false` i gra uzywa tekstury proceduralnej. Inne formaty (32-bit, RLE) sa odrzucane.

Gdzie: `Texture.cpp` -> `loadBMP() (sprawdzenie bpp != 24 || compression != 0)`

P: Dlaczego w BMP zamieniasz R z B?

O: Bo BMP zapisuje piksele w kolejnosci BGR (niebieski, zielony, czerwony), a OpenGL chce RGB. Przy kopiowaniu biore `raw[src+2]` (B) na pozycje R i `raw[src+0]` (R) na pozycje B. G zostaje w srodku.

Gdzie: `Texture.cpp` -> `loadBMP() (petla pixels[dst+0]=raw[src+2] itd.)`

P: Czemu odwracasz wiersze obrazu?

O: BMP z dodatnia wysokoscia trzyma wiersze od dolu do gory (pierwszy wiersz pliku to dol obrazu). OpenGL oczekuje od gory. Wiec `srcRow = topDown ? row : (h-1-row)` - dla zwyklego BMP czytam od konca. Ujemna wysokosc w BMP oznacza juz top-down i wtedy nie odwracam.

Gdzie: `Texture.cpp` -> `loadBMP() (srcRow = topDown ? row : h-1-row)`

P: Co to jest rowStride / padding w BMP?

O: Kazdy wiersz BMP jest dopelniany do wielokrotnosci 4 bajtow. `rowStride = (w*3+3) & ~3` to faktyczna dlugosc wiersza w pliku (z dopelnieniem). Bez uwzglednienia tego obraz by sie 'przesuwal' skosnie, bo zle bym liczyl poczatek kazdego wiersza.

Gdzie: `Texture.cpp` -> `loadBMP() (rowStride = (w*3+3) & ~3)`

P: Skad wiesz, gdzie w pliku zaczynaja sie piksele?

O: Z naglowka pliku (14 bajtow) czytam `dataOffset` (4 bajty od pozycji 10) - to przesuniecie do danych pikseli. Robie `fseek(f, dataOffset)` zanim zaczne czytac piksele. Dzieki temu dziala nawet jak naglowek ma niestandardowy rozmiar.

Gdzie: `Texture.cpp` -> `loadBMP() (dataOffset, fseek)`

Tekstury - generatory proceduralne

P: Jak działa generateCheckerboard (szachownica)?

O: Dla każdego piksela licze które pole szachownicy: $(x/\text{tileSize} + y/\text{tileSize}) \% 2$. Parzyste = kolor 1, nieparzyste = kolor 2. $\text{tileSize} = \text{size}/\text{tileCount}$. To czysta arytmetyka na indeksach pikseli, bez żadnego szumu.

Gdzie: Texture.cpp -> generateCheckerboard()

P: Czym jest value noise i FBM w teksturach proceduralnych?

O: Value noise: losowe wartości w rogach całkowitej kraty, wewnątrz interpolowane płynnie (smoothstep). FBM (Fractional Brownian Motion) sumuje kilka oktafów takiego szumu o coraz wyższej częstotliwości i niższej amplitudzie - daje naturalny, samopodobny wzór. Używam w drewnie, marmurze, ceglach, suficie.

Gdzie: Texture.cpp -> valueNoise2D(), fbm2D() (anonimowy namespace)

P: Jak generujesz drewno (słoje)?

O: Licze odległość piksela od środka tekstury, dodaje do niej zaburzenie z fbm (turbulencja). Z tej odległości robie pile ($\text{dist} \cdot \text{rings}$, część ułamkowa) - to daje koncentryczne kręgi. Interpoluje kolor między ciemnym słojem a jasnym tłem wg pozycji w kręgu.

Gdzie: Texture.cpp -> generateWood()

P: Jak generujesz cegły?

O: Dzielę teksturę na rzędy (rowHeight) i cegły ($\text{brickWidth} = 2 \cdot \text{rowHeight}$). Co drugi rząd przesuwam o połowę cegły (naprzemienny układ). Tam gdzie jest fuga (mortarPx) maluje kolor zaprawy, reszta to cegła z drobną wariacją koloru (hash per cegła + mikroszum). To daje realistyczny mur.

Gdzie: Texture.cpp -> generateBricks()

P: Jak generujesz marmur?

O: Licze turbulencje (suma modułów szumu o malejącej amplitudzie), potem $\text{zyl} = \sin((x + \text{turbulencja} \cdot \text{siła}) \cdot 2\pi)$. Moduł i pierwiastek (pow 0.5) wyodrębia przejścia. To daje wzór falujących żył typowy dla marmuru.

Gdzie: Texture.cpp -> generateMarble()

P: Co to są mipmapy i jak je liczysz?

O: To pomniejszone kopie tekstury (1/2, 1/4...). Gdy obiekt jest daleko, OpenGL używa mniejszej wersji - inaczej tekstura migocze. Liczę je ręcznie w petli: każdy poziom to średnia z bloków 2x2 poprzedniego poziomu, wgrywana przez glTexImage2D z numerem poziomu (level).

Gdzie: Texture.cpp -> uploadToGPU() (petla while $\text{mipW} > 1 \ || \ \text{mipH} > 1$)

P: Dlaczego klasa Texture ma usunięty konstruktor kopiujący?

O: Bo trzyma uchwyt GL (textureId). Kopia miała by ten sam id - i destruktor zwolniłby teksturę dwa razy (glDeleteTextures na tym samym id), co jest błędem. Dlatego = delete na kopiowaniu; teksturę trzymam przez shared_ptr.

Gdzie: Texture.h -> Texture(const Texture&) = delete;

Targets - cele

P: Po co pula celów zamiast tworzenia sfer na bieżąco?

O: Tworzenie i niszczenie obiektów OpenGL co chwile jest kosztowne i ryzykowne. Lepiej raz stworzyć MAX_TARGETS (3) sfery i tylko je chować (setVisible false) i pokazywać. To wzorzec 'object pool'. Każdy aktywny cel ma nIndex wskazujący którą sferę go reprezentuje.

Gdzie: Targets.cpp -> initTargetPool(); ShootingGallery.h -> targetNodePool_

P: Dlaczego ruch celu liczysz sinusem?

O: Sinus daje gładki ruch tam i z powrotem (ping-pong) bez skoków na końcach. $pos = basePos + moveAxis * moveRange * \sin(totalTime * moveSpeed + movePhase)$. Dodatkowo lekkie kołysanie w pionie (drugi sinus na Y).

Gdzie: Targets.cpp -> currentTargetPos()

P: Po co każdy cel ma losową fazę (movePhase)?

O: Żeby cele w jednej fali nie ruszały się idealnie zsynchronizowane (co wyglądałoby sztucznie). Losowa faza przesuwa sinus każdego celu, więc poruszają się niezależnie.

Gdzie: Targets.cpp -> spawnWave() (movePhase = distR(rng_)*6.28); currentTargetPos()

P: Czemu ta sama funkcja liczy pozycje przy rysowaniu i przy strzale?

O: Żeby strzał trafiał dokładnie tam, gdzie cel jest w danej klatce. Gdybym liczył pozycję inaczej do rysowania, a inaczej do kolizji, celowanie byłoby niesprawiedliwe - widziałbym cel gdzie indziej niż jest 'naprawdę'. currentTargetPos jest wołane i w onUpdate (render) i w onShoot.

Gdzie: Targets.cpp -> currentTargetPos(); użycie w Gameplay.cpp onUpdate/onShoot

P: Jak losujesz rozmiar fali (1-3 cele)?

O: Losuje liczbę r w [0,1). Jeżeli $r < 0.5$ -> 1 cel, $r < 0.85$ -> 2 cele, inaczej 3. Czyli najczęściej 1, czasem 2, rzadko 3. Potem dla każdego celu losuje pozycję i tryb ruchu.

Gdzie: Targets.cpp -> spawnWave() (waveSize_ = $r < 0.5 ? 1 : \dots$)

P: Jak unikasz pojawienia się celu w ścianie albo w przeszkodzie?

O: Losuje pozycje i sprawdzam 3 warunki: dystans od gracza ($>MIN$), czy nie w przeszkodzie (`isInsideObstacle` z marginesem na CAŁY zakres ruchu), czy nie za blisko innego celu z fali. Jeżeli któryś warunek niespełniony, losuje ponownie - do 60 prob. Po 60 stawiam cel statyczny w bezpiecznym miejscu (fallback).

Gdzie: `Targets.cpp` -> `spawnWave()` (petla `attempt < 60`); `isInsideObstacle` z `Obstacles.cpp`

P: Czemu margines przy spawnie uwzględnia zakres ruchu celu?

O: Bo cel się porusza. Gdybym sprawdzał tylko pozycje startową, ruchomy cel mógłby wjechać w kolumnę podczas oscylacji. $Margin = TARGET_RADIUS + moveRange + \text{zapas}$ gwarantuje, że cały tor ruchu jest wolny od przeszkód.

Gdzie: `Targets.cpp` -> `spawnWave()` (`margin = TARGET_RADIUS + t.moveRange + 0.25`)